

Middleware в Express.js

Промежуточная обработка HTTP-запросов

Результаты занятия

- ✓ Объяснять роль middleware в цикле request-response
- ✓ Создавать собственные middleware-функции
- ✓ Использовать параметры req, res и next
- ✓ Управлять передачей запроса через next()
- ✓ Понимать влияние порядка подключения middleware
- ✓ Применять middleware на уровне приложения, пути и Router
- ✓ Использовать встроенные middleware Express

Учебные вопросы

Понятие **middleware**

Место middleware в
цикле обработки
HTTP-запроса

Область действия **middleware**

Application-level,
route-level и Router-
level

Создание собственных **middleware**

Логирование,
дополнение req и
предварительная
проверка запроса

next() и порядок выполнения

Передача
управления между
функциями

Встроенные **middleware Express**

Обработка JSON,
форм, текста и
статических файлов



Учебный вопрос 1. Понятие middleware

Что происходит между получением запроса и отправкой ответа

Маршрут — не единственный этап обработки

- HTTP-запрос не обязательно сразу передаётся конечному обработчику маршрута.
- Перед выполнением основной логики сервер может проверить запрос или дополнить его данными.
- Можно записать информацию о запросе в журнал, проверить заголовки или разобрать JSON.
- Одни и те же действия часто нужны сразу для многих маршрутов.
- В Express такие промежуточные действия реализуются с помощью `middleware`.

Что такое middleware

- Middleware — функция, которая получает доступ к объектам `req` и `res` и может участвовать в обработке запроса до формирования окончательного ответа.
- Middleware выполняется во время `request-response cycle`.
- Функция может выполнить собственный программный код.
- Она может изменить объекты `req` или `res`.
- Она может завершить обработку и отправить ответ.
- Либо передать управление следующей функции через `next()`.

Что может делать middleware

● Выполнять код

Например, записывать время и адрес каждого входящего запроса.

● Изменять req и res

Добавлять данные, которые понадобятся следующим обработчикам.

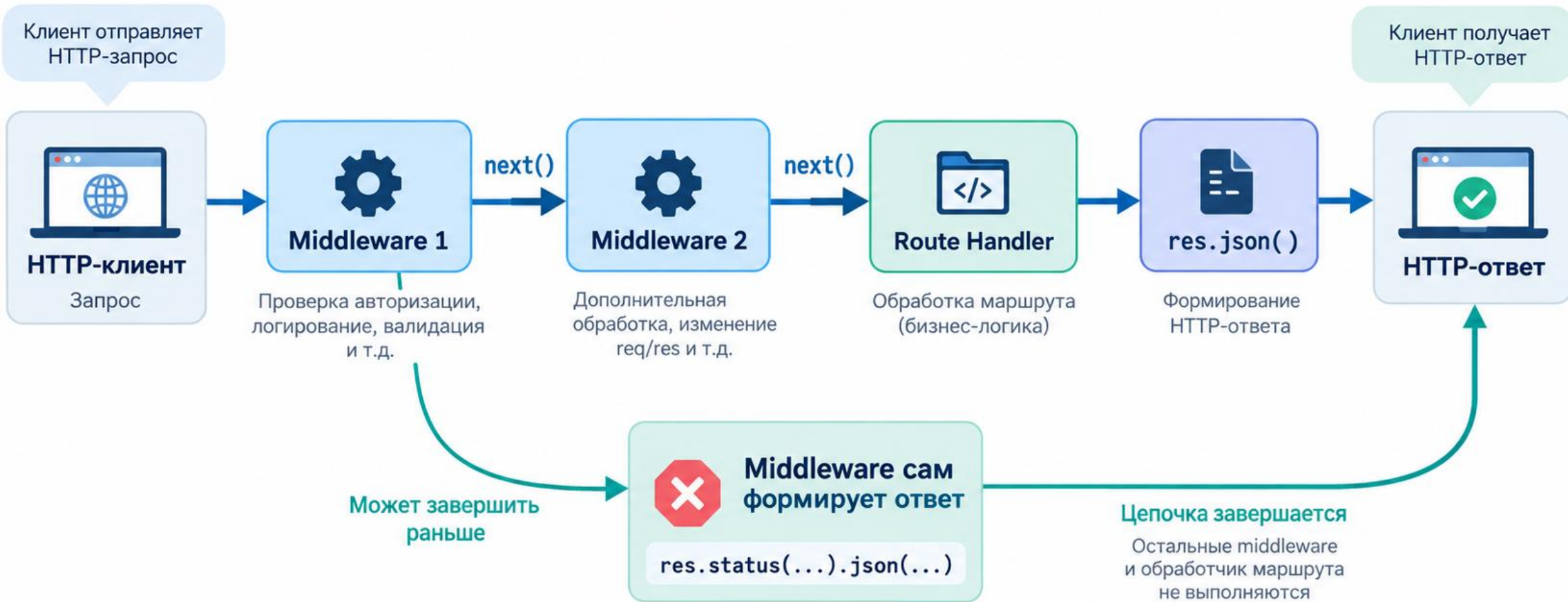
● Завершать запрос

Отправлять ответ клиенту без перехода к следующему обработчику.

● Передавать управление

Вызывать `next()` и продолжать движение запроса по цепочке.

Место middleware в цикле request-response



Middleware может: выполнить код, изменить req/res, вызвать `next()` или завершить запрос.

Три параметра middleware-функции

```
function middleware(req, res, next) {  
  // промежуточная обработка  
}
```

Что важно

- req — объект HTTP-запроса.
- res — объект HTTP-ответа.
- next — функция передачи управления следующему middleware или обработчику.
- Имена параметров req и res являются соглашением, а next важно использовать по назначению.

Первый собственный middleware

```
app.use((req, res, next) => {  
  console.log(req.method, req.url);  
  next();  
});
```

Что важно

- `app.use()` подключает middleware к приложению.
- Функция выполняется при поступлении запроса.
- `req.method` содержит HTTP-метод.
- `req.url` содержит URL текущего запроса.
- `next()` передаёт управление следующему элементу цепочки.

Middleware и конечный обработчик

Middleware

- Выполняет промежуточную работу
- Часто используется для многих маршрутов
- Может вызвать `next()`
- Может изменить `req` или `res`
- Может самостоятельно завершить запрос

Route handler

- Выполняет основную операцию `endpoint`
- Связан с конкретным маршрутом
- Обычно формирует окончательный ответ
- Например, получает товар или создаёт заказ
- Часто находится в конце цепочки



Учебный вопрос 2. `next()` и порядок выполнения

Как запрос проходит через последовательность `middleware`

Для чего нужен next()

```
app.use((req, res, next) => {  
  console.log("Middleware 1");  
  next();  
});  
  
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

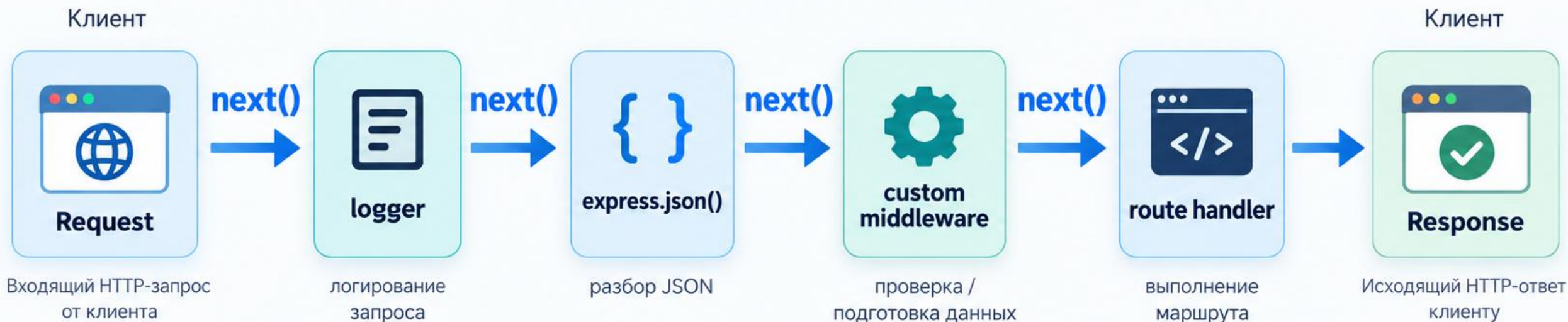
Что важно

- Сначала выполняется middleware.
- После вызова next() Express продолжает обработку запроса.
- Далее может выполняться другое middleware или маршрут.
- Без next() управление дальше автоматически не передаётся.

Больше
чем код
→ возможности

Цепочка middleware и вызов `next()`

Каждый middleware выполняет свою задачу и вызывает `next()`, чтобы передать управление дальше.



`next()` передаёт управление следующему middleware или обработчику маршрута.

Что произойдёт без next()

- Middleware может не отправить ответ и одновременно не вызвать next().
- В этом случае Express не перейдёт к следующему обработчику.
- Маршрут не будет выполнен.
- Клиент продолжит ожидать завершения запроса.
- Это одна из типичных ошибок при написании middleware.

Главная идея

Middleware должно либо завершить request-response cycle, либо передать управление дальше.

Middleware без next()

Запрос: GET /api/products

```
app.use((req, res, next) => {  
  console.log("Request received");  
  // next() отсутствует  
});  
  
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

Результат

Маршрут GET /api/products не будет достигнут.

Пояснение

- Первое middleware получает запрос, но не завершает его и не передаёт управление дальше.

Middleware может завершить запрос

```
app.use((req, res, next) => {  
  const maintenance = true;  
  
  if (maintenance) {  
    return res.status(503).json({  
      message: "Service temporarily unavailable"  
    });  
  }  
  
  next();  
});
```

Что важно

- Middleware не обязано всегда вызывать `next()`.
- Если условие выполнено, оно может сразу отправить ответ.
- После `res.json()` переход к маршруту больше не требуется.
- `return` помогает явно завершить выполнение текущей функции.

Два варианта завершения middleware

Продолжить обработку

- Выполнить промежуточную работу
- Не отправлять окончательный ответ
- Вызвать `next()`
- Передать запрос дальше

Остановить обработку

- Обнаружить условие для остановки
- Сформировать HTTP-статус
- Отправить ответ через `res`
- Не вызывать `next()`

Порядок подключения имеет значение

- Express обрабатывает middleware и маршруты в порядке их регистрации.
- Middleware, объявленное раньше маршрута, может обработать запрос до него.
- Middleware, объявленное после маршрута, не выполнится для запроса, который уже получил ответ.
- Поэтому глобальные middleware обычно подключаются до routers.
- Порядок формирует фактическую цепочку обработки.

Последовательность выполнения

```
app.use((req, res, next) => {  
  console.log("1. Logger");  
  next();  
});  
  
app.use(express.json());  
  
app.use((req, res, next) => {  
  console.log("2. Request prepared");  
  next();  
});  
  
app.use("/api/products", productRoutes);
```

Порядок
имеет значение

Почему порядок middleware **важен**

Express выполняет middleware и маршруты в порядке регистрации.

Маленький
порядок —
большие возможности

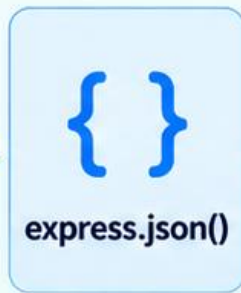


Правильно

Сначала обрабатываем тело запроса, затем маршруты.



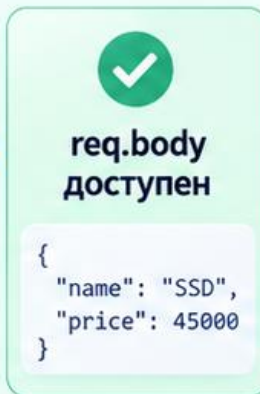
логирование
запроса



разбор JSON



маршрут получает
req.body

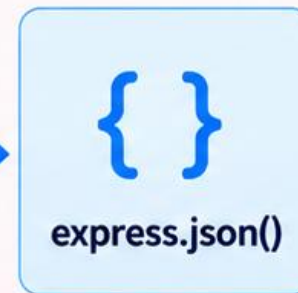


Неправильно

Маршрут выполняется раньше, чем разбирается тело запроса.



маршрут пытается
читать **req.body**



слишком поздно



Сначала middleware обработки тела запроса,
потом маршруты, которые используют **req.body**.



Учебный вопрос 3. Область действия middleware

Глобальная обработка, отдельный путь и конкретный Router

Application-level middleware

```
app.use((req, res, next) => {  
  console.log(`${req.method} ${req.url}`);  
  next();  
});
```

Что важно

- Middleware подключено к основному объекту app.
- Путь в app.use() не указан.
- Поэтому функция будет запускаться для входящих запросов приложения.
- Так удобно реализовывать общее логирование.

Middleware только для определённого пути

```
app.use("/api/products", (req, res, next) => {  
  console.log("Products request");  
  next();  
});
```

Что важно

- Первый аргумент `app.use()` задаёт mount path.
- Middleware применяется к запросам, проходящим через `/api/products`.
- Запросы к другим частям приложения не используют эту функцию.
- Это позволяет ограничивать область действия middleware.

Уровни подключения middleware

Способ	Область действия	Пример
<code>app.use(middleware)</code>	Приложение	Общее логирование
<code>app.use("/api/products", middleware)</code>	URL-префикс	Обработка запросов товаров
<code>router.use(middleware)</code>	Один Router	Обработка routes ресурса
<code>router.get("/", middleware, handler)</code>	Один маршрут	Проверка перед handler

Middleware для конкретного маршрута

```
function logRequest(req, res, next) {  
  console.log("Products requested");  
  next();  
}  
  
router.get("/", logRequest, (req, res) => {  
  res.json(products);  
});
```

Что важно

- Маршрут может получать несколько функций-обработчиков.
- Сначала выполняется logRequest.
- next() передаёт управление следующей функции маршрута.
- Последняя функция формирует JSON-ответ.

Router-level middleware

- Middleware можно подключить непосредственно к Express Router.
- Для этого используется `router.use()`.
- Такое middleware действует только внутри конкретного Router.
- Например, Product Router и Order Router могут иметь разные промежуточные проверки.
- Это сохраняет модульность структуры проекта.

Middleware внутри Product Router

```
const router = express.Router();

router.use((req, res, next) => {
  console.log("Product Router");
  next();
});

router.get("/", getProducts);
router.get("/:id", getProductById);
```

Что важно

- `router.use()` применяется к маршрутам данного Router.
- `GET /api/products` проходит через middleware.
- `GET /api/products/15` также проходит через него.
- Маршруты других routers не затрагиваются.

Области действия middleware в Express

Разные типы middleware работают в разных областях: приложение, URL-префикс, Router и отдельный маршрут

HTTP-клиенты

Отправляют запросы к вашему приложению

GET /api/products

GET /api/categories

GET /api/orders

GET /about



Express Application (app.js)

Обработывает все входящие HTTP-запросы



1. Глобальные middleware (application-level)

app.use(middleware) — выполняются для всех запросов

```
app.use(logger);
app.use(express.json());
```

/api/products

Product Router



2. Middleware с префиксом

app.use("/api/products", ...) работает только для /api/products

```
app.use("/api/products",
  authMiddleware);
```



3. Router-level middleware

router.use() — только внутри productRoutes

```
router.use(checkProducts);
```



4. Route-level middleware

Только для конкретного маршрута

```
router.post("/",
  validateProduct, createProduct);
```

Маршруты (routes)

GET / — список товаров
GET /:id — товар по id
POST / — создать товар

/api/categories

Category Router

router.use()

Только внутри categoryRoutes

```
router.use(checkCategories);
```

Маршруты (routes)

GET / — список категорий
GET /:id — категория по id
POST / — создать категорию

/api/orders

Order Router

router.use()

Только внутри orderRoutes

```
router.use(checkOrders);
```

Маршруты (routes)

GET / — список заказов
GET /:id — заказ по id
POST / — создать заказ

Другие маршруты

Без отдельного Router

Например:

GET /
GET /about
GET /contact

1

Application-level middleware

- Подключается через app.use()
- Работает для всех запросов приложения
- Используется для логирования, разбора JSON и т. п.

2

Middleware с префиксом

- Подключается через app.use("/path")
- Работает только для указанного URL-префикса
- Применяется ко всем маршрутам этого пути

3

Router-level middleware

- Подключается через router.use()
- Работает только внутри одного Router
- Позволяет задавать общую логику для всех routes этого ресурса

4

Route-level middleware

- Указывается в конкретном маршруте
- Работает только для выбранного endpoint
- Используется для проверок, валидации и других задач перед обработчиком



Ключевая идея: Express выполняет middleware в порядке их регистрации и применяет их только в пределах соответствующей области. Это позволяет гибко управлять обработкой запросов и сохранять модульную структуру приложения.



Один HTTP-запрос может пройти через несколько уровней middleware, но только те, которые соответствуют его пути.



Учебный вопрос 4. Встроенные middleware Express

Готовые функции для типовых задач обработки запросов

Встроенные middleware

express.json()

Разбирает входящие HTTP-запросы с JSON payload.

express.urlencoded()

Разбирает данные формы в формате application/x-www-form-urlencoded.

express.static()

Отдаёт клиенту статические файлы из указанного каталога.

express.text() / raw()

Позволяют принимать текстовое содержимое или необработанный Buffer.

express.json()

```
app.use(express.json());

app.post("/api/products", (req, res) => {
  console.log(req.body);
  res.status(201).json(req.body);
});
```

Что важно

- `express.json()` является middleware.
- Оно анализирует JSON-тело входящего запроса.
- После успешного разбора данные доступны через `req.body`.
- Middleware подключается до маршрутов, которые используют тело запроса.

express.urlencoded()

```
app.use(express.urlencoded({ extended: true }));  
  
app.post("/form", (req, res) => {  
  res.json(req.body);  
});
```

Что важно

- Используется для URL-encoded данных.
- Такой формат традиционно применяется HTML-формами.
- Разобранные значения также становятся доступны через `req.body`.
- В REST API с JSON чаще используется `express.json()`.

express.static()

```
app.use(express.static("public"));
```

Что важно

- Middleware позволяет отдавать статические файлы.
- Каталог public может содержать HTML, CSS, JavaScript и изображения.
- Если public содержит файл logo.png, Express может вернуть его по HTTP.
- Для полноценного REST API static используется реже, но полезен для веб-приложений.

Какой встроенный middleware выбрать

Middleware	Входные данные	Типичный результат
<code>express.json()</code>	application/json	req.body как объект
<code>express.urlencoded()</code>	URL-encoded form data	req.body с полями формы
<code>express.text()</code>	text/plain	req.body как строка
<code>express.raw()</code>	Бинарные данные	req.body как Buffer
<code>express.static()</code>	Запрос файла	Статический файл



Учебный вопрос 5. Создание собственных middleware

Выносим повторяющуюся обработку запроса в отдельные функции

Middleware для логирования запросов

```
function logger(req, res, next) {  
  console.log(  
    new Date().toISOString(),  
    req.method,  
    req.originalUrl  
  );  
  next();  
}
```

Что важно

- Функция имеет стандартные параметры req, res, next.
- Дата позволяет определить время запроса.
- req.method показывает HTTP-метод.
- req.originalUrl содержит исходный URL.
- После логирования запрос продолжает обработку.

Добавляем данные в req

```
function addRequestTime(req, res, next) {  
  req.requestTime = new Date().toISOString();  
  next();  
}  
  
app.use(addRequestTime);
```

Что важно

- Middleware может добавлять собственные свойства к req.
- Следующие обработчики смогут использовать requestTime.
- Так данные можно подготовить один раз до выполнения маршрутов.
- После изменения req вызывается next().

Используем данные middleware в маршруте

Запрос: GET /api/status

```
app.get("/api/status", (req, res) => {  
  res.json({  
    status: "ok",  
    requestTime: req.requestTime  
  });  
});
```

```
{  
  "status": "ok",  
  "requestTime": "2026-09-  
16T10:30:00.000Z"  
}
```

Пояснение

- Значение requestTime было создано middleware до запуска обработчика маршрута.

Middleware может проверить условие

```
function checkApiKey(req, res, next) {  
  const apiKey = req.get("x-api-key");  
  
  if (apiKey !== "college-demo") {  
    return res.status(403).json({  
      message: "Access denied"  
    });  
  }  
  
  next();  
}
```

Что важно

- Middleware читает HTTP-заголовок x-api-key.
- При неправильном значении обработка завершается статусом 403.
- При успешной проверке вызывается next().
- Это учебный пример проверки условия, а не полноценная система аутентификации.

Применяем проверку только к одному маршруту

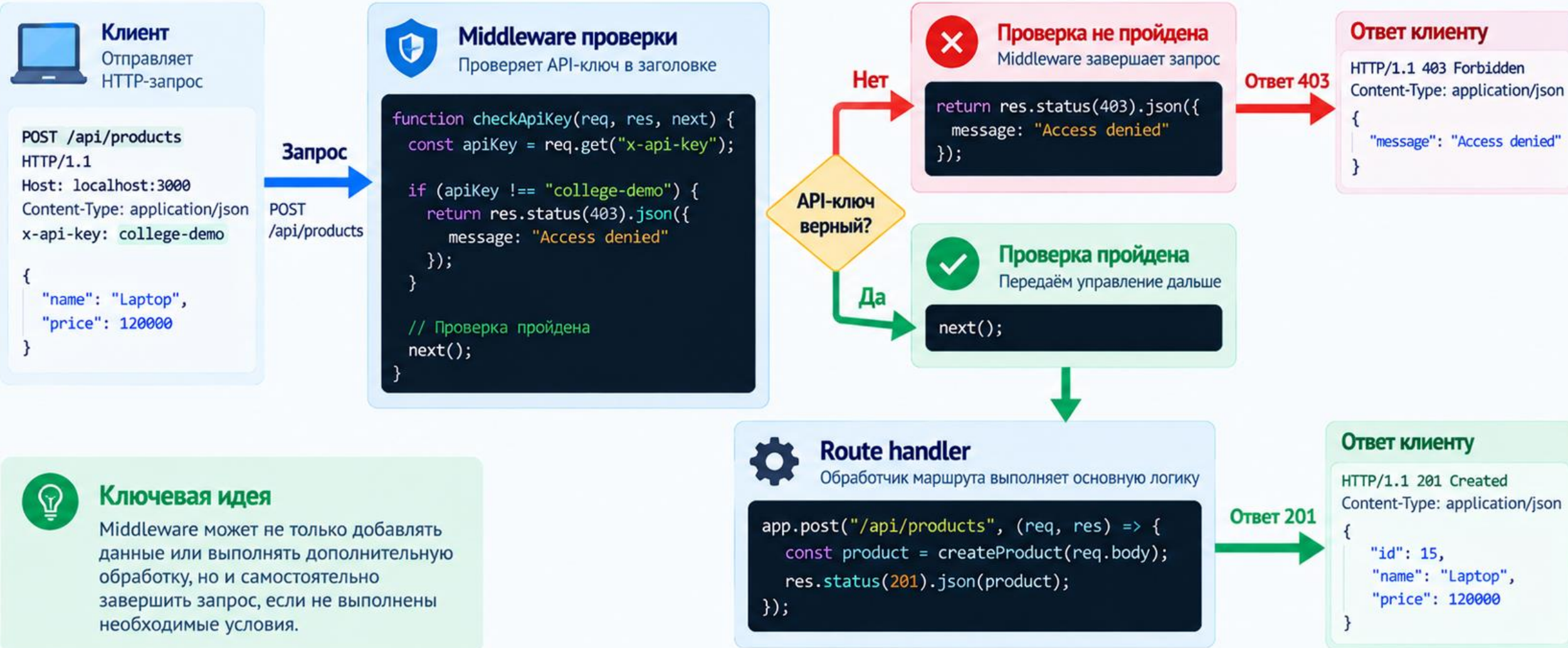
```
router.post(
  "/",
  checkApiKey,
  (req, res) => {
    res.status(201).json(req.body);
  }
);
```

Что важно

- Сначала Express запускает `checkApiKey`.
- Если проверка не пройдена, `handler` не выполняется.
- Если `middleware` вызывает `next()`, управление переходит к `handler`.
- Так маршрут получает дополнительный этап обработки без изменения основной логики.

Middleware как фильтр перед обработчиком

Middleware может проверить запрос и решить: продолжать обработку или сразу вернуть ответ



Ключевая идея

Middleware может не только добавлять данные или выполнять дополнительную обработку, но и самостоятельно завершить запрос, если не выполнены необходимые условия.



Итог: Middleware работает как фильтр: пропускает запрос дальше (next()) или останавливает его и возвращает ответ.



Это позволяет реализовывать аутентификацию, проверки прав доступа, валидацию данных и другие механизмы защиты.

Именованные middleware удобно переиспользовать

- Анонимный middleware удобен для небольшого локального действия.
- Повторяющуюся логику лучше оформить именованной функцией.
- Одну функцию можно подключить к нескольким маршрутам или routers.
- Большие middleware можно вынести в отдельные модули.
- Это уменьшает дублирование кода и делает назначение функций понятнее.

Выносим logger в отдельный файл

```
export default function logger(req, res, next) {  
  console.log(  
    req.method,  
    req.originalUrl  
  );  
  
  next();  
}
```

Подключаем собственный middleware

```
import logger from "../middleware/logger.js";  
  
const app = express();  
  
app.use(logger);  
app.use(express.json());  
app.use("/api/products", productRoutes);
```

Что важно

- Middleware импортируется как обычный ES Module.
- logger подключается глобально через `app.use()`.
- После него подключается обработка JSON.
- Затем подключаются routers.
- Последовательность отражает порядок обработки запроса.

Существуют и сторонние middleware

- Express можно расширять пакетами из npm.
- Например, middleware может реализовывать логирование, CORS, загрузку файлов или работу с cookies.
- Сторонний middleware обычно устанавливается как зависимость и подключается через `app.use()`.
- Перед использованием стороннего пакета необходимо изучить его документацию.
- На следующих занятиях мы будем использовать middleware для конкретных задач приложения.

Error-handling middleware — особый тип

```
app.use((err, req, res, next) => {  
  res.status(500).json({  
    message: "Internal server error"  
  });  
});
```

Что важно

- Middleware для обработки ошибок отличается сигнатурой и принимает четыре параметра: `err`, `req`, `res`, `next`.
- Такой middleware предназначен для централизованной обработки ошибок.
- Он обычно размещается после обычных маршрутов и middleware.
- Подробно обработку ошибок рассмотрим в отдельной теме.

Мини-практика

Добавьте в Internet Shop API несколько middleware и проследите порядок их выполнения.

Задание

- Создать глобальный logger для метода и URL
- Создать middleware addRequestTime и добавить req.requestTime
- Подключить express.json() до маршрутов
- Создать middleware checkApiKey для POST /api/products
- Проверить запрос с правильным и неправильным x-api-key
- Переставить middleware и проследить изменение порядка выполнения

Критерии проверки

- Все запросы фиксируются logger
- В обработчике доступно req.requestTime
- JSON доступен через req.body
- POST без правильного ключа возвращает 403
- POST с правильным ключом доходит до обработчика
- Студент может объяснить роль next()

Контрольные вопросы

1. Что называется middleware в Express.js?
2. Какие три основных параметра получает обычная middleware-функция?
3. Какие действия может выполнять middleware?
4. Для чего используется функция `next()`?
5. Что произойдёт, если middleware не отправит ответ и не вызовет `next()`?

6. Почему порядок подключения middleware имеет значение?
7. Чем application-level middleware отличается от Router-level middleware?
8. Для чего используется `express.json()`?
9. Как middleware может передать дополнительные данные следующему обработчику?
10. Чем error-handling middleware отличается от обычной middleware-функции?

Итоги занятия

1

Express-приложение представляет собой последовательность middleware и обработчиков.

2

Middleware получает доступ к `req`, `res` и функции `next`.

3

Функция может изменить запрос, выполнить дополнительную работу, завершить ответ или передать управление дальше.

4

`next()` используется для перехода к следующему элементу цепочки.

5

Порядок подключения middleware определяет порядок обработки запроса.

6

Middleware можно подключать глобально, к определённому пути, Router или отдельному маршруту.

7

Express содержит встроенные middleware, включая `express.json()`, `express.urlencoded()` и `express.static()`.

8

Собственные middleware позволяют вынести повторяющуюся обработку запросов из маршрутов.